§ 1. Интерфейсы и стоимость операций

Структура данных начинается с обещания пользователю: какие операции допустимы, что они возвращают и сколько ресурсов могут потребовать. Массив, дерево и связный список — способы выполнить это обещание. Одно и то же представление бывает удачным для одного набора операций и неудобным для другого.

После этой главы вы сможете отличать стоимость отдельной операции от стоимости последовательности, доказывать амортизированные оценки методом потенциалов и реализовать очередь с минимумом. Предполагается знакомство с массивами, указателями, рекурсией и базовыми контейнерами C++17.

1.1. Сначала контракт, затем контейнер

Рассмотрим последовательность целых чисел. Нужны чтение и изменение элемента по индексу, добавление и удаление последнего элемента. Назовём этот интерфейс динамическим массивом. Пока мы ничего не сказали о размещении памяти. Список тоже хранит последовательность, но доступ к произвольному индексу обычно требует пройти через предыдущие элементы.

Контракт включает не только названия методов. Надо определить индексацию, допустимость пустой структуры, поведение ошибочных вызовов и требования к устойчивости ссылок. В этой книге индексы начинаются с нуля; диапазон $[l, r)$ включает l, но не включает r. Пустой диапазон разрешён. Извлечение из пустой очереди — ошибка, а не молчаливое возвращение нуля: ноль может быть обычным элементом.

Требование	Массив	Связный список
Прочитать элемент по индексу	Постоянное число адресных операций	До линейного числа переходов
Вставить после известного узла	Возможен сдвиг хвоста	Перепривязка ссылок
Хранить элементы подряд	Да	Обычно нет
Перебрать все элементы	Линейное время, хорошая локальность	Линейное время, переходы по указателям

Последняя строка показывает ограниченность одной асимптотики. Две реализации могут иметь одинаковую оценку, но по-разному использовать кеш процессора. Сначала доказываем корректность и порядок роста, затем измеряем константы на характерной нагрузке.

Связь с видео. В записи ССД от 01.09.2021 около 15:19 на доске появляется `DS Interface`, а около 30:39 — размер, ёмкость и `push_back`. Сохраняем этот ход объяснения: сначала требование к операциям, затем представление. Числа в примерах ниже подобраны для учебника.

1.2. Что именно мы считаем

В модели машинных слов чтение ячейки, сравнение и сложение помещающихся в слово целых чисел считаем операциями постоянной стоимости. Это предположение надо проверять: сумма n чисел может не помещаться в тип элемента. В примерах с суммами используем `long long`; входные ограничения должны гарантировать, что все промежуточные результаты представимы. Переполнение знакового целого в C++ не даёт корректного «вычисления по модулю».

Размер входа также требует определения. Для массива естественный параметр — число элементов n. Для одного большого целого число его битов равно $\lfloor \log_2 n \rfloor + 1$, если $n > 0$. Цикл из n итераций линейный по числовому значению, но экспоненциальный по длине записи числа. В этой главе параметры массивов — числа элементов, а целочисленная арифметика ограничена машинным словом.

Различим три вида гарантий.

- **Худший случай одного вызова:** максимум стоимости по всем допустимым состояниям данного размера.
- **Амортизированная оценка:** оценка общей стоимости серии операций с учётом начального состояния.
- **Ожидаемая оценка:** математическое ожидание относительно явно заданной случайности — входов или внутренних случайных решений.

Амортизация не требует случайного входа. Если противник выбирает каждую следующую операцию, доказанная оценка последовательности всё равно действует. Для средней стоимости на случайных входах необходимо задать распределение; равномерность не возникает автоматически.

Например, при q добавлениях в растущий массив отдельный вызов может копировать почти все ранее добавленные элементы. Это не противоречит линейному времени всех q добавлений. Утверждения относятся к разным объектам: одному вызову и всей истории.

1.3. Динамический массив: дорогая операция и дешёвая история

Пусть `size` — число занятых ячеек, `capacity` — число выделенных. Пока `size < capacity`, записываем новый элемент в свободную ячейку. Когда места нет, выделяем массив удвоенной ёмкости, переносим элементы и дописываем новый.

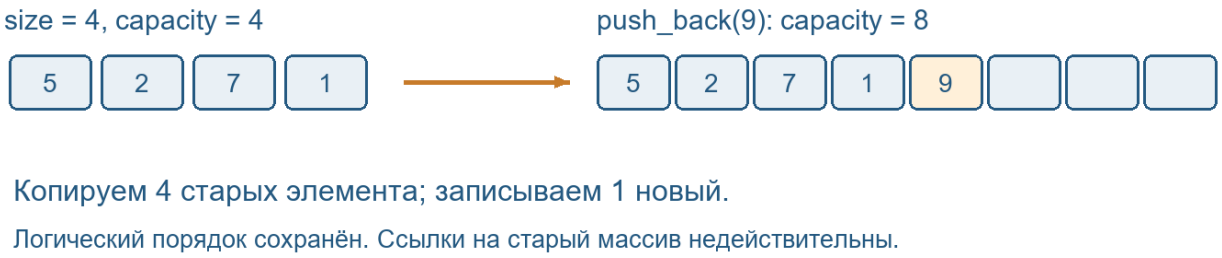


Рисунок 1: Расширение массива меняет ёмкость, но сохраняет логическую последовательность.

Начнём с нулевого размера и первой ёмкости 1. Добавим девять чисел. Копирование потребуется при переходах ёмкости $1 \to 2$, $2 \to 4$, $4 \to 8$, $8 \to 16$. Число скопированных элементов равно $1 + 2 + 4 + 8 = 15$. Ещё девять записей помещают новые элементы. Последний вызов дорогой, но весь процесс содержит только 24 записи или переноса в принятой модели.

Для произвольного q последняя скопированная ёмкость меньше q. Геометрическая сумма всех переносов меньше $2q$. Вместе с q записями получаем менее $3q$ действий, то есть $O(q)$ времени и $O(1)$ амортизированно на добавление. Если выделение блока требует времени, линейного по его длине, суммы выделяемых ёмкостей также образуют геометрическую прогрессию и порядок оценки сохраняется.

Увеличение ёмкости на одну ячейку даёт другую картину: $1 + 2 + \cdots + (q - 1) = \Theta(q^2)$. Причина улучшения при удвоении не в особом числе 2, а в геометрическом росте. Любой фиксированный множитель больше единицы даёт линейную суммарную стоимость, хотя расход памяти и константы меняются.

(c) Aleksey Tolstikov

Пример из записи. Около 45:58 в лекции 01.09.2021 показано копирование массива в больший. Здесь мы продолжаем этот пример точным подсчётом переносов. Последовательность ёмкостей 1, 2, 4, 8, 16 — редакторская иллюстрация идеи, а не дословное восстановление доски.

В C++ `std::vector::push_back` имеет амортизированную константную сложность при обычных предпосылках о стоимости работы с элементом. Стандарт не обещает именно удвоение ёмкости. Перевыделение инвалидирует ссылки и итераторы на элементы. Если размер заранее известен, `reserve` может устранить перевыделения; вызов `reserve(size()+1)` перед каждым добавлением, напротив, способен разрушить геометрический рост.

Дополнение из новой расшифровки, 29:09–30:49. Лектор обсуждает более общую политику `new_capacity = α · capacity + β`: постоянная добавка помогает реже выделять совсем маленькие блоки в начале, а множитель больше единицы обеспечивает геометрический рост в дальнейшем. В записи приводятся добавки 16 или 32 элемента. Это пример из видео; он не задаёт обязательную политику `std::vector`. Для фиксированных $\alpha > 1$, $\beta \geq 0$ прежние ёмкости ограничиваются геометрической прогрессией относительно последней, поэтому суммарные переносы остаются линейными. Округление до целого и минимальная начальная ёмкость должны входить в контракт реализации.

1.4. Метод потенциалов без скрытых предположений

Потенциал $\Phi(S)$ — запас работы, приписанный состоянию структуры S. Пусть фактическая стоимость операции i равна c_i, а амортизированная —

$$\hat{c}_i = c_i + \Phi(S_i) - \Phi(S_{i-1}).$$

Просуммируем равенства. Внутренние потенциалы сокращаются:

$$\sum_{i=1}^{q} c_i = \sum_{i=1}^{q} \hat{c}_i + \Phi(S_0) - \Phi(S_q).$$

Если потенциал неотрицателен, то фактическая сумма не превосходит суммы амортизированных стоимостей плюс начальный потенциал. Поэтому достаточно ограничить $\hat{c}_i$ и отдельно учесть $\Phi(S_0)$. Само по себе название «потенциал» ничего не доказывает: отрицательный конечный запас мог бы скрыть большую стоимость.

Для растущего массива с непустым состоянием выберем $\Phi = 2s - c$, где s — размер, c — ёмкость; для пустого начального состояния зададим $\Phi = 0$. После первого добавления и при дальнейшем удвоении ёмкости $s \leq c < 2s$, поэтому потенциал неотрицателен.

Если расширения нет, фактическая стоимость равна 1, потенциал растёт на 2, значит амортизированная стоимость равна 3. Перед расширением $s = c$ и потенциал равен c. После копирования и добавления $s' = c+1$, $c' = 2c$, потенциал равен 2. Фактическая стоимость $c+1$, поэтому $\hat{c} = (c+1) + (2-c) = 3$. Первый вызов разбирается отдельно и имеет постоянную стоимость.

Это доказательство относится к добавлениям. Если разрешить удаления и немедленное уменьшение ёмкости, надо заново проверить неотрицательность и переходы. Простейшее чередование добавления и удаления около порога может вызвать постоянные дорогие копирования. Типичный способ избежать этого — разные пороги роста и сжатия, например расширение при заполнении и сжатие при четверти заполнения.

1.5. Двоичный счётчик и стек с multipop

У счётчика один инкремент меняет все младшие единицы на нули и ближайший ноль на единицу. Переход $0111 \to 1000$ меняет четыре бита, хотя предыдущие переходы могли быть дешевле. При q инкрементах от нуля бит 0 меняется q раз, бит 1 — $\lfloor q/2 \rfloor$, бит 2 — $\lfloor q/4 \rfloor$. До переполнения фиксированной разрядности сумма меньше $2q$.

То же можно доказать потенциалом: число единичных битов. Если сбрасываются k единиц и устанавливается один ноль, то $c = k + 1$, $\Delta\Phi = 1 - k$, следовательно $\hat{c} = 2$. Для произвольного начального состояния добавляется его потенциал. Поэтому формулировка «любые q инкрементов стоят не больше $2q$» без оговорки о старте неверна.

Переход	Изменено битов	Изменение потенциала	Амортизированная стоимость
0000 → 0001	1	1	2
0001 → 0010	2	0	2
0011 → 0100	3	−1	2
0111 → 1000	4	−2	2

В стеке `multipop(k)` удаляет до k элементов. Один вызов может стоить $\Theta(n)$, но каждый добавленный элемент удаляется не более одного раза. Начиная с пустого стека, суммарное число удалений не превосходит числа добавлений. Это агрегатное доказательство; потенциалом можно взять текущий размер стека.

Эти примеры полезны как шаблон поиска доказательства: найдите объект, который оплачивает дорогую работу и не может оплатить её повторно. Для счётчика это накопленные единицы, для стека — ещё не удалённые элементы, для массива — запас свободной ёмкости, созданный предыдущим расширением.

1.6. Стек с минимумом

Хранить один текущий минимум недостаточно. Если удалить элемент, равный минимуму, нужно узнать минимум оставшегося стека. Вместе с каждым элементом сохраним минимум всего префикса до него. Тогда верхняя пара содержит и значение, и ответ для текущего состояния.

Для добавлений 5, 2, 2, 7 пары будут `(5,5), (2,2), (2,2), (7,2)`. После удаления 7 минимум остаётся 2; после удаления одной двойки он также равен 2. Дубликаты не требуют отдельной логики. Инвариант доказывается индукцией: новая вторая компонента равна минимуму нового значения и предыдущего минимума; удаление восстанавливает уже корректный префикс.

(c) Aleksey Tolstikov

```cpp
class MinStack {
    std::vector<std::pair<long long, long long>> a;
public:
    bool empty() const { return a.empty(); }
    void push(long long x) {
        long long m = empty() ? x : std::min(x, a.back().second);
        a.emplace_back(x, m);
    }
    long long top() const {
        if (empty()) throw std::out_of_range("empty stack");
        return a.back().first;
    }
    long long minimum() const {
        if (empty()) throw std::out_of_range("empty stack");
        return a.back().second;
    }
    void pop() {
        if (empty()) throw std::out_of_range("empty stack");
        a.pop_back();
    }
};
```

На уровне абстрактного стека все действия постоянные. В данной реализации хранилище — `std::vector`, поэтому добавление пары имеет амортизированную константную стоимость, а перевыделение может быть линейным. Чтение и удаление последнего элемента постоянны для нашего простого типа. Дополнительная память на элемент — одно значение минимума.

1.7. Очередь на двух стеках

Входной стек `in` принимает новые элементы. Выходной `out` хранит начало очереди на вершине. Пока `out` не пуст, извлекаем из него. Когда он пуст, переносим в него все элементы из `in`: порядок переворачивается, и самый старый элемент оказывается сверху.

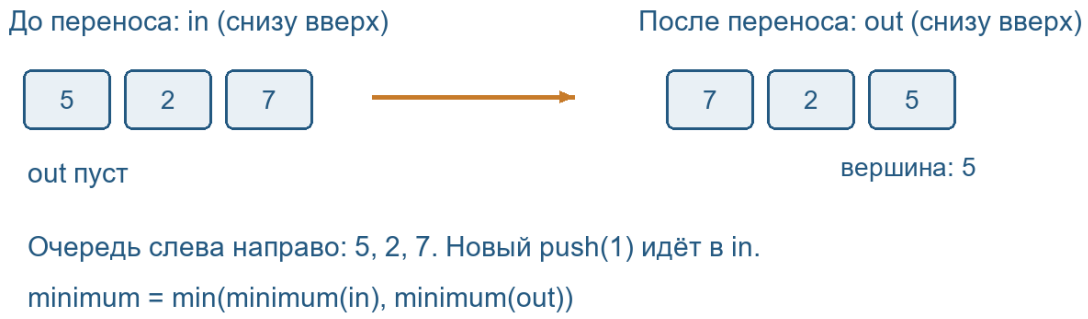


Рисунок 2: Два стека представляют одну очередь; перенос меняет физический порядок, сохраняя логический.

Например, после добавлений 5, 2, 7 входной стек снизу вверх содержит 5, 2, 7. Перенос создаёт выходной стек снизу вверх 7, 2, 5. Добавим 1 во входной стек. Логическая очередь теперь 5, 2, 7, 1; её минимум равен минимуму двух стековых минимумов.

(c) Aleksey Tolstikov

```
class MinQueue {
    MinStack in, out;
    void prepare() {
        if (out.empty()) {
            while (!in.empty()) {
                long long x = in.top();
                out.push(x);
                in.pop();
            }
        }
    }
public:
    bool empty() const { return in.empty() && out.empty(); }
    void push(long long x) { in.push(x); }
    long long front() {
        prepare();
        return out.top();
    }
    void pop() {
        prepare();
        out.pop();
    }
    long long minimum() const {
        if (in.empty()) return out.minimum();
        if (out.empty()) return in.minimum();
        return std::min(in.minimum(), out.minimum());
    }
};
```

Если оба стека пусты, `minimum`, `front` и `pop` заканчиваются исключением из `MinStack`. Мы не назначаем фиктивный минимум пустой очереди. Операция `front` тоже может инициировать перенос, поэтому её худшая стоимость линейная.

Каждый элемент один раз попадает во входной стек, не более одного раза переносится в выходной и один раз удаляется. За q вызовов суммарно выполняется $O(q)$ стековых операций. Амортизация внутренних векторов также даёт $O(q)$ суммарной работы, поэтому две оценки не конфликтуют. При моделировании стековых операций как единичных можно взять потенциал $2|in|$: перенос k элементов выполняет $2k$ действий и уменьшает потенциал на $2k$.

Важно переносить элементы **только в пустой** выходной стек. Иначе новые элементы окажутся перед старыми и нарушат FIFO. Очередь с минимумом не является решением для дека с удалениями с обоих концов: такой интерфейс требует другого баланса и отдельного доказательства.

1.8. Проверка и самостоятельная работа

Для очереди эталоном служит `std::deque`: добавление справа, удаление слева, минимум прямым просмотром. После каждой случайной операции сравнивайте начало и минимум. Проверяйте пустую очередь, единственный элемент, одинаковые значения, отрицательные числа, несколько переносов и длинную серию чтений без изменений. Случайные проверки дополняют инвариант, но не заменяют его.

Для временной сложности тесты должны считать переносы, а не только секунды. На маленьких входах таймер больше измеряет накладные расходы. При удвоении числа операций линейный счётчик переносов объясняет поведение лучше одного удачного замера.

Практика. Реализуйте очередь с минимумом по приведённому контракту и независимую проверку на прямой модели. Для углубления рассмотрите iRunner 4185 «Добавить один много раз» и 4022 «Минимум в списке». У второй задачи операции разрешены на обоих концах; это сознательное усложнение, а не прямое применение листинга.

Что переносится в следующие главы. Мы будем выбирать между дешёвыми обновлениями и дешёвыми запросами, явно считать объём дополнительной памяти и доказывать сохранение инварианта после каждой операции. Дерево отрезков даст логарифмические гарантии для обоих типов действий.

1.9. Разбор: минимум в скользящем окне

Пусть измерения поступают последовательно, а после каждого нового значения нужен минимум последних k измерений. При $k = 3$ и потоке $[5, 2, 7, 1, 4]$ полные окна имеют минимумы $[2, 1, 1]$. Это применение очереди с минимумом: добавляем новое значение, удаляем самое старое, если размер превысил k, и читаем минимум, когда окно заполнено. Числа этого разбора выбраны для учебника.

Получено	Очередь после удаления лишнего	Ответ
5	5	Окно ещё не заполнено
2	5, 2	Окно ещё не заполнено
7	5, 2, 7	2
1	2, 7, 1	1
4	7, 1, 4	1

Внешний алгоритм хранит размер очереди отдельным счётчиком. При добавлении увеличивает его, при удалении уменьшает. Контракт требует $1 \le k \le n$; при другом соглашении надо отдельно определить, существуют ли ответы для неполного окна. Каждое измерение добавляется ровно один раз и удаляется не более одного раза, поэтому весь проход занимает $O(n)$, а занятые элементы — $O(k)$ памяти. Даже если один перенос стеков стоит $\Theta(k)$, он не происходит заново для каждого окна.

Есть другая структура — монотонная очередь индексов. Из её хвоста удаляют индексы элементов, не меньших нового, а из головы — вышедшие за левую границу окна. Оставшиеся значения возрастают. Почему можно забыть удалённый из хвоста элемент? Новый элемент не больше него и покинет окно позже. Во всех будущих окнах, где старый мог бы быть полезен, новый будет не хуже.

Это доказательство использует специальную последовательность запросов: границы окна движутся только вправо. Произвольный запрос к прошлому диапазону разрушил бы аргумент. Очередь на двух стеках хранит все живые элементы и реализует обычный FIFO; монотонная очередь хранит только кандидатов на минимум. Их оценки одинаковы по порядку, но интерфейсы и основания корректности различаются. Чтобы выбрать структуру, сначала определите допустимые операции.

1.10. Углубление: почему сжатие требует зазора

Предположим, массив растёт вдвое при переполнении и сжимается вдвое, как только заполнение падает до половины. Возьмём состояние размера $c/2 + 1$ и ёмкости c. Удаление одного элемента вызывает сжатие до $c/2$ с переносом $c/2$ элементов. Следующее добавление требует снова расшириться до c. Повторяя эти два действия, получаем линейную стоимость почти каждой операции, хотя логический размер меняется всего на единицу.

Исправим правило: сжимаем только при размере не больше четверти ёмкости. После сжатия вдвое массив заполнен примерно наполовину. До следующего расширения нужны многие добавления, а

до следующего сжатия — многие удаления. Этот промежуток позволяет накопить плату за переносы. Разные пороги переключения называют гистерезисом.

Для ёмкости $c \geq 1$ и размера s рассмотрим потенциал

$$\Phi(s, c) = \begin{cases} 2s - c, & s \geq c/2, \\ c/2 - s, & s < c/2. \end{cases}$$

Он неотрицателен и равен нулю при половинном заполнении. Вставка без расширения меняет потенциал не более чем на 2, удаление без сжатия — не более чем на 1. Рост из полного состояния разобран выше: уменьшение потенциала оплачивает перенос старых элементов.

Для сжатия удобно разобрать точный порог: до удаления размер $c/4 + 1$, после него $c/4$, новая ёмкость $c/2$. До операции потенциал равен $c/4 - 1$, после — нулю. Если стоимость удаления и копирования равна $1 + c/4$, амортизированная стоимость равна 2. Малые ёмкости и округление целых размеров меняют лишь константу; практически задают минимальную ёмкость и не сжимают ниже неё.

Полученная гарантия относится к нашей политике контейнера. Она не означает, что метод `pop_back` у `std::vector` автоматически уменьшает выделенную память. Разделяйте логический размер, занятую ёмкость и контракт конкретной библиотеки. Для очереди со стеками, реализованными векторами, резерв памяти может зависеть от максимального достигнутого размера, а не от числа элементов в текущий момент.

Самостоятельный эксперимент. Считайте отдельно число операций пользователя и число переносов элементов. Сравните увеличение ёмкости на единицу, удвоение и удвоение со сжатием при половине заполнения. В последнем случае используйте описанное чередование, а не случайный вход. Эксперимент должен иллюстрировать уже доказанную границу, а не заменять её измерением одного удачного запуска.

1.11. Практика к главе

В расширенной подборке приложения A: №4185 «Добавить один много раз» (основная); №4022 «Минимум в списке» (углубление). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

1.12. Вопросы для самопроверки

1. Сформулируйте контракт очереди с минимумом: допустимые вызовы, результат каждого метода и поведение пустой структуры.
2. Почему линейная стоимость одного `push_back` совместима с амортизированной константной?
3. Сколько переносов старых элементов потребуется для 17 вставок в пустой массив с начальной ёмкостью 1 и удвоением при заполнении?
4. Что изменится при увеличении ёмкости на 10 ячеек?
5. Получите телескопическую формулу метода потенциалов и назовите нужное ограничение на конечный потенциал.
6. Сколько битов меняется при переходе $31 \to 32$? Как это согласуется с оценкой последовательности?
7. Почему начальное состояние нельзя игнорировать в анализе счётчика?
8. Проследите очередь после `push(5), push(2), pop(), push(1), front()`.
9. Приведите пример нарушения FIFO при переносе в непустой выходной стек.
10. Почему сжатие при половинном заполнении может дать линейную стоимость почти каждой операции? Постройте историю и объясните роль порога в одну четверть.